UE23CS342BA4 - Generative AI and its Applications

Handson - unit 2 Date: 17/02/2026

Name: Bhuvi Prashanth	SRN: PES2UG23CS130	Section: B

Notebook-1[LangChain]

LangChain is useful as a unified interface for different language models, making it easy to switch providers with minimal code changes.
Models that process text as tokens, not words, and that cost and memory limits depend on token usage.Understanding tokens is important because they directly affect pricing and context window limits.

Temperature

Setting temperature to different values

```
[4]
✓ 11s
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")

--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
Run 2: An idea is a thought, concept, or suggestion that is formed or exists in the mind.
```

```
[5]
✓ 9s
print("--- CREATIVE (Temp=1) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")

--- CREATIVE (Temp=1) ---
Run 1: An idea is a thought, concept, or plan formed in the mind.
Run 2: An idea is a thought or concept formed in the mind, often as a basis for understanding, action, or creation.
```

```
[6]
✓ 8s
print("--- CREATIVE (Temp=2) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")

--- CREATIVE (Temp=2) ---
Run 1: An idea is a mental concept, plan, or suggestion that is formed in the mind as a product of thought.
Run 2: An idea is a mental concept, thought, plan, or suggestion conceived by the mind.
```

Observation

This example shows that temperature controls the creativity and randomness of the model's responses.
At temperature 0, the outputs are consistent and identical across runs, showing deterministic behavior.As the temperature increases (1 and 2), the responses become more varied and expressive.Higher temperature leads to more creative but less predictable outputs.

Strings Vs Messages

```python
from langchain_core.messages import SystemMessage, HumanMessage

# Scenario: Make the AI rude.
messages = [
    SystemMessage(content="You are a rude teenager. You use slang and don't care about grammar."),
    HumanMessage(content="What is the capital of France?")
]

response = llm.invoke(messages)
print(response.content)
```

```
Ugh, Paris. Duh. Like, can't u just google that yourself? So dumb.
```

Observation

This example shows that LLMs respond differently depending on the role of the message (System vs Human).
The SystemMessage acts like a controller that defines the AI's personality, tone, and behavior.
When the system role makes the AI "rude," the response changes accordingly, even though the question is simple.
This demonstrates how powerful system instructions are in shaping model output.

Prompt templates

```python
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a translator. Translate {input_language} to {output_language}."),
    ("human", "{text}")
])

# We can check what inputs it expects
print(f"Required variables: {template.input_variables}")
```

```
Required variables: ['input_language', 'output_language', 'text']
```

Observation

This code defines a structured chat prompt using system and human roles, making the interaction more organized.
By printing `template.input_variables`, we can verify what inputs are required before invoking the chain.
This helps prevent errors by ensuring all necessary variables are provided during execution.

LCEL

```
[15]
2s
# Define the chain once
chain = template | llm | parser

# Invoke the whole chain
print(chain.invoke({"topic": "Octopuses"}))

Here's a fun one:

Did you know octopuses have **three hearts**? Two of them pump blood through their gills, and the third one circulates it to the rest of their body! This is also why they have blue blood!
```

Observation

This example demonstrates how LCEL uses the pipe (|) operator to create clean and modular chains.
The output of one component (template) is passed directly to the next (LLM, parser), improving readability.
It highlights composability, allowing easy swapping of models or adding intermediate steps like a spellchecker.
This approach makes AI workflows more structured, maintainable, and scalable.

Assignment

Create a chain that:

1. Takes a movie name.
2. Asks for its release year.
3. Calculates how many years ago that was (You can try just asking the LLM to do the math).

Try to do it in **one line of LCEL**.

```
[18]
2s
chain = ChatPromptTemplate.from_template(
    "What year was {movie} released and how many years ago was that from 2026?"
) | ChatGoogleGenerativeAI(model="gemini-2.5-flash")

print(chain.invoke({"movie": "Titanic"}).content)

The movie Titanic was released in **1997**.

From 2026, that will be **29 years ago** (2026 - 1997 = 29).
```

Notebook-2
Co-star framework

```python
# The Task: Reject a candidate for a job.
task = "Write a rejection email to a candidate."

print("--- LAZY PROMPT ---")
print(llm.invoke(task).content)
```

```
--- LAZY PROMPT ---
Here are a few options for a rejection email, ranging from a standard template to one for a candidate who interviewed. Choose the one that best fits your situation.

---

**Option 1: Standard Rejection (No Interview)**

This is suitable for candidates who applied but were not selected for an interview.

**Subject: Update on Your Application for [Job Title] at [Company Name]**

Dear [Candidate Name],

Thank you for your interest in the [Job Title] position at [Company Name] and for taking the time to submit your application.

We received a large number of highly qualified applications for this role. While your qualifications are impressive, we have decided to move forward with other candidates whose profiles were a closer match for the specific requirements of

We appreciate you considering [Company Name] as a potential employer and wish you the best of luck in your job search and future endeavors.

Sincerely,

[Your Name]
[Your Title]
[Company Name]
[Company Website (Optional)]

---

**Option 2: Rejection After Interview(s)**

This option acknowledges the time and effort the candidate put into the interview process.

**Subject: Update Regarding Your Application for [Job Title] at [Company Name]**

Dear [Candidate Name],
```

Observation

The comparison shows that a lazy prompt produces generic, overly long outputs with multiple template options.
In contrast, the CO-STAR framework structures the prompt with clear context, objective, tone, audience, and format.
This structured approach results in more focused, relevant, and controlled responses.
It highlights how better prompt design directly improves output quality and precision.

Hallucination v/s creativity

```python
structured_prompt = """
# Context
You are an HR Manager at a quirky startup called 'RocketBoots'.

# Objective
Write a rejection email to a candidate named Bob.

# Constraints
1. Be extremely brief (under 50 words).
2. Do NOT say 'we found someone better'. Say 'the role changed'.
3. Sign off with 'Keep flying'.

# Output Format
Plain text, no subject line.
"""

print("--- STRUCTURED PROMPT ---")
print(llm.invoke(structured_prompt).content)
```

```
--- STRUCTURED PROMPT ---
Hi Bob,

Thank you for your interest in RocketBoots.

We appreciate you taking the time to interview. However, the role's requirements have changed significantly since your application. We wish you the best in your job search.

Keep flying.
```

Observation

This example shows how the model fills in missing details by predicting the most likely reason for rejection.
Since no specific reason was provided, it generated a common explanation (“the role changed”), demonstrating pattern completion.
This behavior is not necessarily hallucination but creative inference based on learned templates.
It highlights the importance of providing explicit facts to control model output precisely.

Assignment

Write a structured prompt to generate a **Python Function**.

- **Context:** You are a Senior Python Dev.
- **Objective:** Write a function to reverse a string.
- **Constraint:** It must use recursion (no slicing `[::-1]`).
- **Style:** Include detailed docstrings.

[5] 14s

```python
structured_prompt = """
# Context
You are a Senior Python Developer.

# Objective
Write a Python function to reverse a string.

# Constraints
1. The function must use recursion.
2. Do NOT use slicing (no [::-1]).
3. Do NOT use built-in reverse functions.

# Style
1. Include detailed docstrings.
2. Follow clean and professional coding standards.
3. Add example usage at the bottom.

# Output Format
Return only valid Python code.
"""
print("--- STRUCTURED PROMPT ---")
print(llm.invoke(structured_prompt).content)
```

```
--- STRUCTURED PROMPT ---
```python
def reverse_string_recursive(s: str) -> str:
    """
    Reverses a given string using recursion.

    This function takes a string and returns a new string with its characters
    in reverse order. It strictly adheres to the following constraints:
    1.  Must use recursion.
    2.  Does NOT use the `[::-1]` slicing shortcut for reversal.
    3.  Does NOT use built-in reverse functions (e.g., `reversed()`).

    The recursive approach works by breaking down the problem:
    -   The base case is an empty string or a string with a single character,
        which is already reversed.
    -   For a string with multiple characters, the function takes the first
        character, recursively reverses the *rest* of the string (from the
        second character onwards), and then appends the first character
        to the end of the recursively reversed substring. This effectively
        moves the first character to the last position in the final reversed string.

    Args:
        s: The input string to be reversed.

    Returns:
        The reversed string.

    Examples:
        >>> reverse_string_recursive("hello")
        "olleh"
        >>> reverse_string_recursive("Python")
        "nohtyP"
        >>> reverse_string_recursive("")
        ""
        >>> reverse_string_recursive("a")
        "a"
```
```

Zero-shot to few-shot

Observation

1. **Zero-shot prompting** gives the model only the task instruction, without examples. The output depends entirely on the model's prior training and may be more generic or inconsistent. Zero-shot is faster and more concise, while few-shot increases token usage but improves reliability and control over the output.
2. **Few-shot prompting** provides example inputs and outputs, helping the model understand the expected pattern, format, and style more clearly.Few-shot prompts usually produce more accurate, structured, and task-aligned responses compared to zero-shot prompts.

Advanced templates

Observation

Engineering vs Training

1. Hard prompts (prompt engineering) modify only the input text, making them fast, cheap, and ideal for prototyping.
2. Soft prompts (fine-tuning) modify model weights, requiring data, time, and higher cost but offering deeper domain specialization.
3. Prompt engineering is flexible and iterative, while fine-tuning is better for long-term behavioral or domain-specific changes.

Dynamic Few-Shot Prompting

1. When many examples exist, a selector (e.g., semantic search) chooses the most relevant ones to fit within the context window.
2. Injecting only the top-matching examples improves relevance and efficiency.
3. Dynamic few-shot prompting enhances response quality while controlling token usage and scalability.

Notebook-3

Chain of Thought (CoT)

1. Encourages the model to generate step-by-step reasoning before producing the final answer.
2. Improves accuracy for math, logic, coding, and multi-step problem solving.
3. Easy to implement using prompts like “Let’s think step by step.”
4. Moderate increase in token usage compared to simple prompting.

Tree of Thought (ToT)

1. Explores multiple reasoning paths instead of a single linear chain.
2. Evaluates different branches and selects the best solution.
3. Useful for strategic planning, decision-making, and complex problem solving.
4. Higher computational cost and latency due to multiple reasoning attempts.

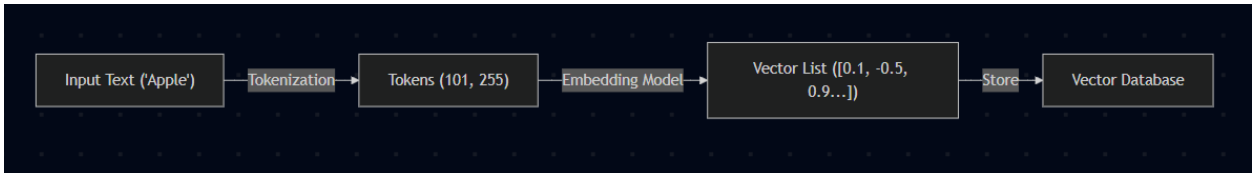
Graph of Thought (GoT)

1. Extends reasoning beyond trees by allowing interconnected reasoning nodes.
2. Enables combining and refining ideas from multiple branches.
3. Suitable for research synthesis, creative writing, and idea generation.
4. Most computationally expensive but offers flexible and rich reasoning structures.

Comparison table

Method	Structure	Use cases	Cost/Latency
Simple Prompt	Input → Output	Facts, summaries	⭐ Low
Chain of thought	Input → Steps → Output	Math, Logic, Debugging	⭐⭐ Med
Tree of thought	Input → 3x Branches → Judge	Strategy, Brainstorming	⭐⭐⭐ High
Graph of thought	Input → Split → Aggregate	Creative Writing, Research	⭐⭐⭐⭐ V. High

Notebook-4



RAG

1. RAG addresses LLM limitations like knowledge cutoff and hallucination by retrieving relevant external data before generating a response.
2. Embeddings convert text into numerical vectors, enabling machines to understand semantic similarity rather than raw words.
3. Vector stores (e.g., FAISS) efficiently store and search these embeddings for relevant information retrieval.
4. Indexing techniques (Flat, IVF, HNSW, PQ) improve search efficiency and scalability in large vector databases.

Vector and calculating similarity

1. Words are represented as high-dimensional vectors (e.g., 384 dimensions in MiniLM), where each dimension encodes a semantic feature.
2. These numerical values are not random; they capture meaning and allow similar words to have similar vector representations.
3. Cosine similarity measures how close two vectors are by calculating the angle between them.
4. A similarity score closer to 1 indicates strong semantic similarity, while 0 means unrelated and −1 indicates opposite meaning.

Naive RAG

1. Naive RAG follows a simple Retrieval → Augmentation → Generation pipeline, similar to an open-book test where the model first finds relevant information before answering.
2. A knowledge base provides external (non-parametric) memory, reducing hallucinations by grounding responses in retrieved documents.
3. FAISS is used for efficient vector indexing and similarity search, enabling fast retrieval of the most relevant documents.
4. LCEL connects the retriever, prompt, and LLM into a structured chain, ensuring the retrieved context is properly injected before generating the final answer.

Indexing algorithms

1. As the number of vectors increases (e.g., from 10 to 100 million), similarity search becomes computationally expensive, creating a scalability challenge.
2. FAISS is designed to efficiently handle large-scale vector searches by optimizing indexing and retrieval strategies.
3. There is a trade-off triangle between speed (query time), accuracy (recall), and memory usage (RAM), meaning improving one often affects the others.
4. Different indexing algorithms are chosen based on application needs, balancing performance, precision, and resource constraints.

Flat index [brute force]

1. As the number of vectors increases (e.g., from 10 to 100 million), similarity search becomes computationally expensive, creating a scalability challenge.
2. FAISS is designed to efficiently handle large-scale vector searches by optimizing indexing and retrieval strategies.
3. There is a trade-off triangle between speed (query time), accuracy (recall), and memory usage (RAM), meaning improving one often affects the others.
4. Different indexing algorithms are chosen based on application needs, balancing performance, precision, and resource constraints.

IVF [inverted file index]

1. IVF (Inverted File Index) improves search efficiency by clustering vectors into groups and searching only within the most relevant cluster.
2. Instead of checking all vectors, it first finds the closest centroid and then performs a local search, reducing computation time significantly.
3. The index must be trained before use so it can learn cluster centroids and partition the vector space effectively.
4. IVF offers faster search than Flat Index but may sacrifice some accuracy depending on the number of clusters and search parameters.

HNSW [hierarchical navigable small world]

1. HNSW organizes vectors in a multi-layer graph structure, enabling efficient navigation through nearest neighbors.
2. It uses hierarchical layers, where higher layers act as “express highways” to quickly reach distant regions of the vector space.
3. HNSW offers extremely fast retrieval with high recall compared to other approximate indexing methods.
4. However, it requires more memory since it must store graph connections (edges) between nodes.

PQ [product quantization]

1. Product Quantization (PQ) reduces memory usage by compressing vectors into smaller sub-vectors and approximating their values.
2. It is a lossy compression technique, meaning some accuracy is sacrificed in exchange for reduced storage and faster search.
3. PQ is especially useful when working with very large datasets (e.g., billions of vectors) under RAM constraints.
4. It significantly improves scalability but may lower recall compared to exact or graph-based indexing methods.

Algorithm	Approach	Advantages/disadvantages
Flat Index	Brute Force	100% Accuracy but very slow
IVF	Clustering	Fast, skips unrelated data
HNSW	Graph based	Extremely fast, high RAM usage
PQ	Compression	Massive memory savings (10x)

Assignment

```
User Input
    ↓
Router (temperature=0)
    ↓
Category
    ↓
Expert System Prompt
    ↓
LLM Response (temperature=0.7)
```

```
[2]
✓ 2s
import os
import getpass

os.environ["GROQ_API_KEY"] = getpass.getpass("Enter your Groq API Key: ")

Enter your Groq API Key: ··· ··· ···

[14]
✓ 0s
from groq import Groq

client = Groq(api_key=os.environ["GROQ_API_KEY"])

BASE_MODEL = "llama-3.1-8b-instant"
```

1. The API key is securely collected using `getpass()`, ensuring it is not displayed openly in the notebook.
2. The key is stored in an environment variable (`GROQ_API_KEY`), which is a secure and standard practice for handling credentials.
3. The Groq client is initialized using this environment variable, enabling authenticated API access.
4. The base model (`llama-3.1-8b-instant`) is explicitly defined, making it easy to switch models later if needed. [mistral was decommissioned]

```python
[15]
MODEL_CONFIG = {
    "technical": {
        "system_prompt": """You are a Technical Support Expert.
Be precise, logical, and code-focused.
Provide debugging steps and example fixes."""
    },
    "billing": {
        "system_prompt": """You are a Billing Support Specialist.
Be empathetic and policy-driven.
Explain charges clearly and outline refund steps."""
    },
    "general": {
        "system_prompt": """You are a helpful Customer Support Assistant.
Handle general inquiries professionally."""
    }
}
```

```python
[16]
def route_prompt(user_input: str) -> str:
    routing_prompt = f"""
Classify the following text into one of these categories:
[technical, billing, general]

Return ONLY the category name.

Text: {user_input}
"""

    response = client.chat.completions.create(
        model=BASE_MODEL,
        messages=[
            {"role": "system", "content": "You are a strict classification engine."},
            {"role": "user", "content": routing_prompt}
        ],
        temperature=0
    )

    return response.choices[0].message.content.strip().lower()
```

```python
[17]
def process_request(user_input: str) -> str:
    category = route_prompt(user_input)

    if category not in MODEL_CONFIG:
        category = "general"

    system_prompt = MODEL_CONFIG[category]["system_prompt"]

    response = client.chat.completions.create(
        model=BASE_MODEL,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_input}
        ],
        temperature=0.7
    )

    return f"[Routed to: {category.upper()}]\n\n{response.choices[0].message.content}"
```

1. Defining our experts in configuration dictionary.
2. The `route_prompt` function uses a deterministic setting (`temperature=0`) to ensure consistent and reliable intent classification.
3. The router strictly enforces returning only the category name, reducing ambiguity in downstream processing.
4. The `process_request` function acts as an orchestrator by selecting the appropriate system prompt based on the routed category.
5. Different temperature settings are used strategically — low for routing (accuracy) and higher for expert responses (flexibility and richness).

```
[18]
query = "My python script is throwing an IndexError on line 5."
print(process_request(query))
```

```
[Routed to: TECHNICAL]

To help you debug the issue, I'll need more information. Please provide the following:

1. The complete error message, including the line number and the error itself.
2. The code snippet where the error is occurring (line 5).
3. The relevant code context, such as the variables and lists used before line 5.

Here's a general approach to troubleshoot the issue:

1. Check the index being used:
   - Is the index valid? (e.g., within the bounds of the list or tuple)
   - Are you using a 0-based index (Python uses 0-based indexing)?

2. Verify the data structure:
   - Are you accessing an element of a list or tuple that is shorter than expected?
   - Have you initialized the data structure correctly?

3. Consider using the `len()` function:
   - You can use `len()` to get the length of a list or tuple, and then use that value to access elements safely.

Here's an example of how to safely access elements in a list:

```python
my_list = [1, 2, 3]

# Safe way to access elements
for i in range(len(my_list)):
    print(my_list[i])

# Alternative way using list comprehension
print([my_list[i] for i in range(len(my_list))])
```

If you provide more context and information about the error, I can give you more specific guidance.

To get the code snippet where the error is occurring (line 5), you can run your script in a debugger or use the following Python command:
```python
import debugger
debugger.debug(filename.py)
```
Replace `filename.py` with the name of your Python script.

Please provide more details about your script and the error you're experiencing.
```

How can I install Python libraries? | Load data from Google Drive | Show a

```
[19]
print(process_request("I was charged twice for my subscription."))
```

```
[Routed to: BILLING]

I'm so sorry to hear that you were charged twice for your subscription. I'm here to help you understand what happened and provide you with a solution.

To better assist you, can you please provide me with some information?

1. Your account number or subscription details (if you have them)
2. The dates of the duplicate charges
3. The amounts charged

Once I have this information, I'll review the situation and explain what happened. I'll also provide you with the next steps to resolve the issue.

As a Billing Support Specialist, I want to assure you that we take cases like this seriously and will work with you to get the correct charges adjusted. You won't be charged again for the duplicate amount, and we'll process a refund as soon as possible.

Please rest assured that I'm here to help and will guide you through every step of the process.
```

```
[20]
print(process_request("Hi, how are you?"))
```

```
[Routed to: GENERAL]

I'm doing well, thank you for asking. How can I assist you today? Is there something specific you'd like help with or a question you have?
```

All the queries being correctly routed to the respective experts.We ensured correct routing by combining **deterministic rule-based checks** (for tool queries) with an **LLM-based classifier** (temperature = 0) for other categories.
The router strictly returns only the category name, reducing ambiguity.
The orchestrator then dynamically loads the correct system prompt or tool function based on that category.
This hybrid approach improves accuracy, consistency, and reliability across all query types.

Bonus challenge [adding “Tool use” expert]

```
[28]
✓ 0s
print(process_request("What is the current price of Bitcoin?"))

[Routed to: TOOL]

Current Bitcoin price is $65,432.10 (mock live price)
```

1. **Added a "tool" category** inside `MODEL_CONFIG` to extend the MoE architecture.
2. **Updated the router logic** to include deterministic rule-based detection (checking for keywords like "price" and "bitcoin").
3. **Created a mock function** `get_bitcoin_price()` to simulate fetching real-time data.
4. **Modified the orchestrator (`process_request`)** to bypass the LLM when the category is `"tool"` and directly call the function instead.